

Using Cursors

In this chapter, you'll learn what cursors are and how to use them.

Understanding Cursors

Note

This Chapter Requires MySQL 5 Support for cursors was added to MySQL 5. Therefore, this chapter is applicable to MySQL 5 or later only.

As you have seen in previous chapters, MySQL retrieval operations work with sets of rows known as *result sets*. The rows returned are all the rows (zero or more of them) that match a SQL statement. Using simple `SELECT` statements, there is no way to get the first row, the next row, or the previous 10 rows, for example. There is also no way to process all rows one at a time (as opposed to processing all of them in a batch).

Sometimes there is a need to step through rows forward or backward and one or more at a time. This is what cursors are used for. A cursor is a database query stored on the MySQL server; it is not a `SELECT` statement but the result set retrieved by that statement. Once a cursor is stored, applications can use it to scroll or browse up and down through the data as needed.

Cursors are used primarily by interactive applications in which users need to scroll up and down through screens of data to browse or make changes.

Note

Only in Stored Procedures Unlike most DBMSs, MySQL cursors may only be used within stored procedures (and functions).

Working with Cursors

Using cursors involves several distinct steps:

1. Before a cursor can be used, it must be declared (defined). This process does not actually involve retrieving any data; it merely involves defining the `SELECT` statement to be used.
2. After it is declared, the cursor must be opened for use. This process involves actually retrieving the data using the previously defined `SELECT` statement.
3. With the cursor populated with data, individual rows can be fetched (retrieved) as needed.
4. When it is done, the cursor must be closed.

After a cursor is declared, it can be opened and closed as often as needed. After it is opened, fetch operations can be performed as often as needed.

Creating Cursors

Cursors are created using the `DECLARE` statement (which you saw in Chapter 23, “Working with Stored Procedures”). `DECLARE` names the cursor and takes a `SELECT` statement, complete with `WHERE` and other clauses, if needed. For example, this statement defines a cursor named `ordernumbers` using a `SELECT` statement that retrieves all orders:

► Input

```
DELIMITER //  
CREATE PROCEDURE processorders()  
BEGIN  
    DECLARE ordernumbers CURSOR  
    FOR  
    SELECT order_num FROM orders;  
END //  
DELIMITER ;
```

► Analysis

This stored procedure does not do a whole lot. A `DECLARE` statement is used to define and name the cursor—in this case `ordernumbers`. Nothing is done with the cursor, and as soon as the stored procedure finishes processing, it will cease to exist (as it is local to the stored procedure).

Note

DROP and CREATE As you saw in Chapter 23, to update a stored procedure, you need to use `DROP` on it and then use `CREATE`.

Now that the cursor is defined, it is ready to be opened.

Opening and Closing Cursors

Cursors are opened using the `OPEN CURSOR` statement, like this:

► Input

```
| OPEN ordernumbers;
```

► Analysis

When the `OPEN` statement is processed, the query is executed, and the retrieved data is stored for subsequent browsing and scrolling.

After cursor processing is complete, the cursor should be closed using the `CLOSE` statement, as follows:

► Input

```
| CLOSE ordernumbers;
```

► Analysis

`CLOSE` frees up any internal memory and resources used by the cursor, and so every cursor should be closed when it is no longer needed.

After a cursor is closed, it cannot be reused without being opened again. However, a cursor does not need to be declared again to be used; an `OPEN` statement is sufficient.

Note

Implicit Closing If you do not explicitly close a cursor, MySQL will close it automatically when the `END` statement is reached.

Here is an updated version of the previous example:

► Input

```
| DELIMITER //  
| CREATE PROCEDURE processorders()  
| BEGIN  
|     -- Declare the cursor  
|     DECLARE ordernumbers CURSOR  
|     FOR  
|     SELECT order_num FROM orders;  
|  
|     -- Open the cursor  
|     OPEN ordernumbers;  
|  
|     -- Close the cursor  
|     CLOSE ordernumbers;  
|  
| END//  
| DELIMITER ;
```

 Analysis

This stored procedure declares, opens, and closes a cursor. However, nothing is done with the retrieved data.

Using Cursor Data

After a cursor is opened, each row can be accessed individually using a `FETCH` statement. `FETCH` specifies what is to be retrieved (the desired columns) and where retrieved data should be stored. It also advances the internal row pointer within the cursor so the next `FETCH` statement will retrieve the next row (and not the same one over and over).

The first example retrieves a single row from the cursor (the first row):

 Input

```
DELIMITER //
```

```
CREATE PROCEDURE processorders()  
BEGIN  
  
    -- Declare local variables  
    DECLARE o INT;  
  
    -- Declare the cursor  
    DECLARE ordernumbers CURSOR  
    FOR  
    SELECT order_num FROM orders;  
  
    -- Open the cursor  
    OPEN ordernumbers;  
  
    -- Get order number  
    FETCH ordernumbers INTO o;  
  
    -- Close the cursor  
    CLOSE ordernumbers;  
  
END //
```

```
DELIMITER ;
```

 Analysis

Here `FETCH` is used to retrieve the `order_num` column of the current row (starting at the first row automatically) into a local declared variable named `o`. Nothing is done with the retrieved data.

In the next example, the retrieved data is looped through from the first row to the last:

 Input

```
DELIMITER //
```

```
CREATE PROCEDURE processorders()
```

```

BEGIN

    -- Declare local variables
    DECLARE done BOOLEAN DEFAULT 0;
    DECLARE o INT;

    -- Declare the cursor
    DECLARE ordernumbers CURSOR
    FOR
    SELECT order_num FROM orders;

    -- Declare continue handler
    DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET done=1;

    -- Open the cursor
    OPEN ordernumbers;

    -- Loop through all rows
    REPEAT

        -- Get order number
        FETCH ordernumbers INTO o;

    -- End of loop
    UNTIL done END REPEAT;

    -- Close the cursor
    CLOSE ordernumbers;

END //

DELIMITER ;

```

► Analysis

Like the previous example, this example uses `FETCH` to retrieve the current `order_num` into a declared variable named `o`. Unlike the previous example, the `FETCH` here is within a `REPEAT`, so it is repeated over and over until `done` is true (as specified by `UNTIL done END REPEAT;`). To make this work, the variable `done` is defined with `DEFAULT 0` (false, not done). So how does `done` get set to true when done? The answer is this statement:

```
| DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET done=1;
```

This statement defines a `CONTINUE HANDLER`, which is code that will be executed when a condition occurs. Here it specifies that when `SQLSTATE '02000'` occurs, `SET done=1`. And `SQLSTATE '02000'` is a *not found* condition, and so it occurs when `REPEAT` cannot continue because there are no more rows to loop through.

Caution

DECLARE Statement Sequence There is a specific order in which DECLARE statements, if used, must be issued. Local variables defined with DECLARE must be defined before any cursors or handlers are defined, and handlers must be defined after any cursors. Failure to follow this sequencing will generate an error message.

If you were to call this stored procedure, it would define variables and a CONTINUE HANDLER, define and open a cursor, repeat through all rows, and then close the cursor.

With this functionality in place, you can now place any needed processing inside the loop—after the FETCH statement and before the end of the loop.

Note

REPEAT or LOOP? In addition to the REPEAT statement used here, MySQL also supports a LOOP statement that can be used to repeat code until LOOP is manually exited using a LEAVE statement. In general, the syntax of the REPEAT statement makes it better suited for looping through cursors.

To put this all together, here is one further revision of our stored procedure with cursor, this time with some actual processing of fetched data:

► Input

```
DELIMITER //

CREATE PROCEDURE processorders()
BEGIN
    -- Declare local variables
    DECLARE done BOOLEAN DEFAULT 0;
    DECLARE o INT;
    DECLARE t DECIMAL(8,2);

    -- Declare the cursor
    DECLARE ordernumbers CURSOR
    FOR
    SELECT order_num FROM orders;

    -- Declare continue handler
    DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET done=1;

    -- Create a table to store the results
    CREATE TABLE IF NOT EXISTS ordertotals
        (order_num INT, total DECIMAL(8,2));

    -- Open the cursor
    OPEN ordernumbers;
```

```

-- Loop through all rows
REPEAT

    -- Get order number
    FETCH ordernumbers INTO o;

    -- Get the total for this order
    CALL ordertotal(o, 1, t);

    -- Insert order and total into ordertotals
    INSERT INTO ordertotals(order_num, total)
    VALUES(o, t);

-- End of loop
UNTIL done END REPEAT;

-- Close the cursor
CLOSE ordernumbers;

END //

DELIMITER ;

```

► Analysis

In this example, we've added another variable named `t` (which will store the total for each order). The stored procedure also creates a new table named `ordertotals` on-the-fly (if it does not already exist). This table will store the results generated by the stored procedure. `FETCH` fetches each `order_num` as it did before, and then `CALL` executes another stored procedure (the one created in the previous chapter) to calculate the total with tax for each order (the result of which is stored in `t`). Finally, `INSERT` is used to save the order number and total for each order.

Now the stored procedure can be executed using `CALL`, like this:

► Input

```
CALL processorders();
```

► Analysis

Calling the stored procedure returns no data, but it does create and populate another table that can then be viewed using a simple `SELECT` statement:

► Input

```
SELECT *
FROM ordertotals;
```

► Output

```

+-----+-----+
| order_num | total |
+-----+-----+

```

	20005		158.86	
	20006		58.30	
	20007		1060.00	
	20008		132.50	
	20009		40.78	
+-----+-----+				

And there you have it: a complete working example of stored procedures, cursors, row-by-row processing, and even stored procedures calling other stored procedures.

Summary

In this chapter, you learned what cursors are and why they are used. You also saw examples demonstrating basic cursor use, as well as techniques for looping through cursor results and for row-by-row processing.